

TRABAJO DE INVESTIGACIÓN Y DISEÑO

Arquitectura Híbrida y Planificación DevSecOps para Concesionaria de Motos

Asignatura	Arquitectura multi-cloud
Nombres	Julio Astroza, Adolfo Kliebs
Docente	Felipe Henríquez
Fecha	05-06-26

Índice

Índice	2
1. Resumen Ejecutivo	3
2. Introducción	4
3. Desarrollo	5
3.1. Definición del Proyecto de Software y Contenerización	5
A. Propósito del Software.....	5
B. Arquitectura Desacoplada.....	5
C. Estrategia de Contenedores	5
3.2. Diseño de Arquitectura Híbrida y Multicloud	7
A. Diagrama de Arquitectura	7
B. Entorno Local (On-Premise)	8
C. Entorno Cloud y Alta Disponibilidad.....	8
D. Uso de Servicios Nativos.....	9
3.3. Automatización e Infraestructura como Código (IaC).....	9
A. Aprovisionamiento de Infraestructura	9
B. Gestión de Configuración	10
3.4. Estrategia DevSecOps y Ciberseguridad	10
A. Flujo de Trabajo CI/CD	10
B. Seguridad en el Código (Shift-Left).....	11
C. Bastionado de Red y Protección de Datos	12
D. Gestión de Accesos e Identidad (IAM).....	12
4. Conclusiones	13
5. Bibliografía / Webgrafía.....	14

1. Resumen Ejecutivo

El presente documento detalla el plano arquitectónico para el desarrollo y despliegue de una plataforma web orientada a una concesionaria de motos. El sistema, que incluye un catálogo administrable y un módulo de reserva de horas para servicio técnico, se diseña bajo una arquitectura híbrida y Multicloud.

Utilizando AWS para el cómputo y Azure para la persistencia de datos estáticos y relacionales, la infraestructura se aprovisionará mediante prácticas de Infraestructura como Código (IaC) con Terraform y Ansible. Asimismo, se integra una sólida estrategia DevSecOps para garantizar despliegues continuos, seguros y altamente disponibles.

2. Introducción

En el ciclo de vida del software moderno, la adopción de arquitecturas Multicloud/Híbridas y modelos DevSecOps resulta fundamental para garantizar escalabilidad, seguridad y eficiencia. El objetivo de esta investigación es planificar el despliegue de una aplicación web contenerizada y desacoplada.

A través de este diseño, se busca:

- Comprender los fundamentos de la arquitectura de microservicios.
- Diseñar una topología de alta disponibilidad integrando servicios PaaS.
- Planificar la automatización mediante herramientas de IaC.
- Desarrollar un marco de trabajo DevSecOps que asegure la protección perimetral y el control de accesos en entornos distribuidos.

3. Desarrollo

3.1. Definición del Proyecto de Software y Contenerización

A. Propósito del Software

El proyecto consiste en una aplicación web para una concesionaria de motos. Contará con un sistema de autenticación (Login) exclusivo para administradores, quienes podrán realizar operaciones CRUD sobre el catálogo de motos. Además, incluirá un sistema para que los clientes puedan reservar horas de servicio técnico.

B. Arquitectura Desacoplada

- Separación Lógica: Se implementará una separación estricta entre el Frontend (interfaz de usuario) y el Backend (API y lógica de negocio).
- Justificación Frontend: Se desarrollará con React empaquetado con Vite, elegido por su alto rendimiento y modularidad.
- Justificación Backend: Se desarrollará en Django, por su robusto sistema ORM, seguridad nativa y agilidad para construir operaciones CRUD.

C. Estrategia de Contenedores

Imágenes Base utilizadas:

- Django: imagen python:3.10-slim-bullseye (basada en Debian) para mantener dependencias seguras y ligeras.
- React: arquitectura de dos etapas: node:20-alpine para la construcción y nginx:1.25-alpine para servir la aplicación estática.

Portabilidad: Los contenedores garantizarán que la aplicación funcione de manera idéntica tanto en los entornos de desarrollo locales como en la nube pública, mitigando el problema de dependencias del sistema operativo base.

Versionado de Imágenes Base y Mitigación de Riesgos: Si bien las etiquetas (tags) genéricas como alpine o latest ofrecen agilidad durante la fase de desarrollo, su uso en ambientes de producción constituye una práctica contraria a los principios de DevSecOps. Estas etiquetas son mutables: apuntan a una versión cambiante de la imagen que el proveedor actualiza de forma continua, lo que rompe el principio de reproducibilidad del pipeline, ya que un mismo Dockerfile puede generar artefactos binarios distintos en cada ejecución del flujo de CI/CD, dependiendo del momento exacto de la construcción. Adicionalmente, una actualización no controlada de la imagen base puede introducir cambios incompatibles (breaking changes) o nuevas vulnerabilidades sin que el equipo lo perciba, dificultando la trazabilidad exigida por los escaneos de Aqua Trivy. Por estas razones, se fijan (pin) las versiones a `node:20-alpine` y `nginx:1.25-alpine`, garantizando que cada build sea determinista, auditable y consistente entre los entornos de desarrollo, staging y producción, además de permitir una gestión controlada y planificada de las actualizaciones de seguridad.

D. Descripción de Módulos Funcionales y Casos de Uso

El sistema contempla dos módulos funcionales centrales, cada uno asociado a un actor distinto dentro del flujo de negocio de la concesionaria: el Administrador, responsable de la gestión del catálogo, y el Cliente, responsable de la solicitud de servicios técnicos.

Módulo de Catálogo Administrable: Este módulo permite al actor Administrador, previamente autenticado mediante el sistema de Login, ejecutar operaciones completas de

gestión sobre el catálogo de motocicletas (Crear, Leer, Actualizar y Eliminar — CRUD). Las funcionalidades contempladas son: (1) registrar una nueva motocicleta junto con sus atributos (marca, modelo, cilindrada, precio y stock); (2) editar la información de un modelo existente; (3) eliminar motocicletas descontinuadas del catálogo; y (4) cargar y reemplazar las imágenes representativas de cada producto. Las imágenes son enviadas desde el Frontend (React) hacia el Backend (Django), el cual gestiona su almacenamiento en Azure Blob Storage mediante el SDK correspondiente, devolviendo una URL del recurso que se persiste en Azure Database for PostgreSQL como referencia. Este enfoque desacopla el contenido binario pesado de la base de datos relacional, optimizando el rendimiento de las consultas y reduciendo los costos de almacenamiento en la capa de cómputo.

Módulo de Reserva de Horas: Este módulo habilita al actor Cliente, sin necesidad de privilegios administrativos, para agendar una hora de servicio técnico para su motocicleta. El flujo contempla: (1) selección del tipo de servicio (mantención, revisión técnica o reparación); (2) elección de fecha y bloque horario disponible, validado contra la disponibilidad existente; (3) ingreso de los datos de contacto y de la motocicleta; y (4) confirmación de la reserva. Toda la información transaccional (cliente, fecha, hora y estado de la reserva) es procesada por el Backend en Django y persistida de forma relacional en Azure Database for PostgreSQL, aprovechando las garantías ACID del motor para evitar conflictos de doble reserva sobre un mismo bloque horario.

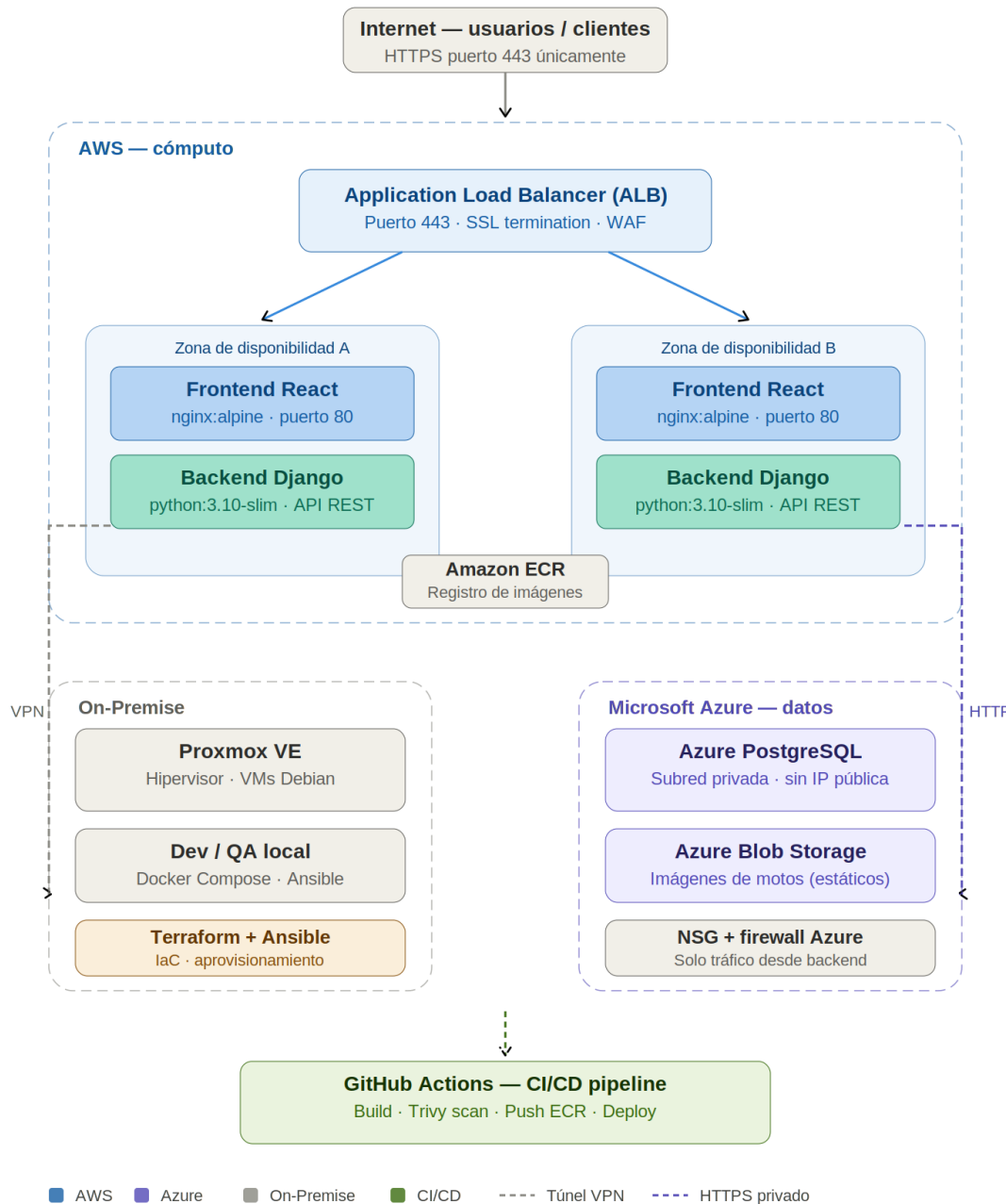
A continuación se presenta el código fuente en notación PlantUML correspondiente al Diagrama de Casos de Uso de ambos módulos, identificando los actores Cliente y Administrador:

```
@startuml
left to right direction
actor Cliente
actor Administrador
rectangle "Sistema Web Concesionaria de Motos" {
    usecase "Iniciar Sesión" as UC0
    usecase "Gestionar Catálogo (CRUD)" as UC1
    usecase "Cargar Imagen a Azure Blob Storage" as UC2
    usecase "Consultar Catálogo de Motos" as UC3
    usecase "Reservar Hora de Servicio Técnico" as UC4
    usecase "Seleccionar Fecha y Horario Disponible" as UC5
    usecase "Confirmar Reserva" as UC6
}
Administrador --> UC0
Administrador --> UC1
UC1 ..> UC2 : <<include>>
Cliente --> UC3
Cliente --> UC4
UC4 ..> UC5 : <<include>>
UC4 ..> UC6 : <<include>>
@enduml
```

Nota: se recomienda renderizar el código anterior mediante un editor compatible con PlantUML (extensión de Visual Studio Code, plugin de IntelliJ o el servidor en línea oficial) e incorporar la imagen resultante (.png o .svg) como figura de esta sección en el informe final.

3.2. Diseño de Arquitectura Híbrida y Multicloud

A. Diagrama de Arquitectura



B. Entorno Local (On-Premise)

- **Nodos Locales:** El entorno de desarrollo y pruebas internas operará On-Premise mediante máquinas virtuales gestionadas por el hipervisor Proxmox VE, ejecutando distribuciones Linux como Debian.

C. Entorno Cloud y Alta Disponibilidad

- **Balanceadores de Carga:** El tráfico hacia los contenedores alojados en AWS será distribuido mediante un Balanceador de Carga de Aplicaciones (ALB), evitando la saturación de los nodos individuales.

- Zonas de Disponibilidad (AZ): Los recursos de cómputo en AWS estarán replicados en múltiples Zonas de Disponibilidad para asegurar redundancia ante caídas de centros de datos.

Especificación de Región y Zonas de Disponibilidad: Con el propósito de estandarizar la topología de despliegue, el cómputo en AWS se aprovisionará en la región `us-east-1` (Norte de Virginia), distribuyendo las instancias de los contenedores entre las zonas de disponibilidad `us-east-1a` y `us-east-1b`. Esta distribución dual de AZ garantiza que, ante la caída o degradación de un centro de datos físico, el Balanceador de Carga de Aplicaciones (ALB) continúe enrutando el tráfico hacia la zona operativa restante, sin interrupción perceptible del servicio. De forma equivalente, los recursos de persistencia en Azure (Azure Database for PostgreSQL y Azure Blob Storage) se aprovisionarán en la región `East US`, geográficamente colindante con `us-east-1`. Esta alineación regional entre ambos proveedores responde a una estrategia de mitigación de latencia cruzada en arquitecturas Multicloud (multi-cloud cross-region latency mitigation): al minimizar la distancia física y el número de saltos de red entre la capa de cómputo (AWS) y la capa de datos (Azure), se reducen los tiempos de respuesta de las consultas a la base de datos y de las operaciones de lectura/escritura sobre el almacenamiento de objetos, evitando que la latencia inter-cloud se convierta en un cuello de botella para la experiencia del usuario final.

D. Uso de Servicios Nativos

- Object Storage: Azure Blob Storage para el almacenamiento de archivos estáticos pesados (imágenes de motos), reduciendo costos de transferencia en la capa de cómputo.
- Base de Datos PaaS: La persistencia de datos (usuarios, reservas y catálogo) se alojará en Azure Database for PostgreSQL, delegando tareas de parcheo, backups y alta disponibilidad al proveedor.

Nota: Se prohíbe estrictamente la instalación de motores de bases de datos en máquinas virtuales locales (IaaS) para el entorno de producción.

3.3. Automatización e Infraestructura como Código (IaC)

A. Aprovisionamiento de Infraestructura

Herramienta: Terraform

Justificación: Terraform es una herramienta agnóstica de nube que permite declarar recursos tanto para AWS (redes, balanceadores) como para Azure (PostgreSQL, Blob Storage) en un mismo ciclo de trabajo, eliminando la necesidad de configuración manual en consola.

Fragmento conceptual — `main.tf` para Azure PostgreSQL:

```
resource "azurerm_postgresql_server" "db_concesionaria" {
  name                = "pg-concesionaria-db"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  sku_name            = "B_Gen5_1"
  version             = "11"
  ssl_enforcement_enabled = true
}
```

Análisis de Factibilidad y Plan de Contingencia: La arquitectura ideal contempla un modelo de IaC completamente agnóstico, en el cual Terraform gestiona de manera unificada los recursos de AWS y Azure dentro de un mismo flujo de trabajo declarativo, tal como se describe en el fragmento conceptual anterior. No obstante, dado que el presente proyecto se desarrolla y despliega sobre el entorno educativo de AWS Academy, es necesario considerar las restricciones inherentes al rol de laboratorio (`LabRole`) asignado a los estudiantes. Dicho rol aplica políticas de IAM estrictas que, en la práctica, bloquean la creación de ciertos recursos globales y la configuración avanzada de VPC, subredes y tablas de enrutamiento a

través de herramientas de aprovisionamiento de terceros como Terraform, al no contar este con los permisos de IAM necesarios para operar fuera del catálogo de servicios habilitado por el laboratorio. Como plan de contingencia, se adopta un modelo híbrido de Infraestructura como Código: AWS CloudFormation se emplea para el aprovisionamiento de la capa de cómputo y red en AWS (VPC, subredes, Balanceador de Carga y contenedores), por ser el mecanismo de despliegue nativo reconocido y permitido por el rol LabRole; mientras que Terraform se mantiene exclusivamente para el aprovisionamiento de los recursos en Azure (Azure Database for PostgreSQL y Azure Blob Storage), donde no existen las mismas restricciones de IAM. Este enfoque preserva el principio de automatización integral del despliegue, adaptándose de manera pragmática a las limitaciones operativas del entorno académico sin comprometer la trazabilidad ni la reproducibilidad de la infraestructura.

B. Gestión de Configuración

Herramienta: Ansible

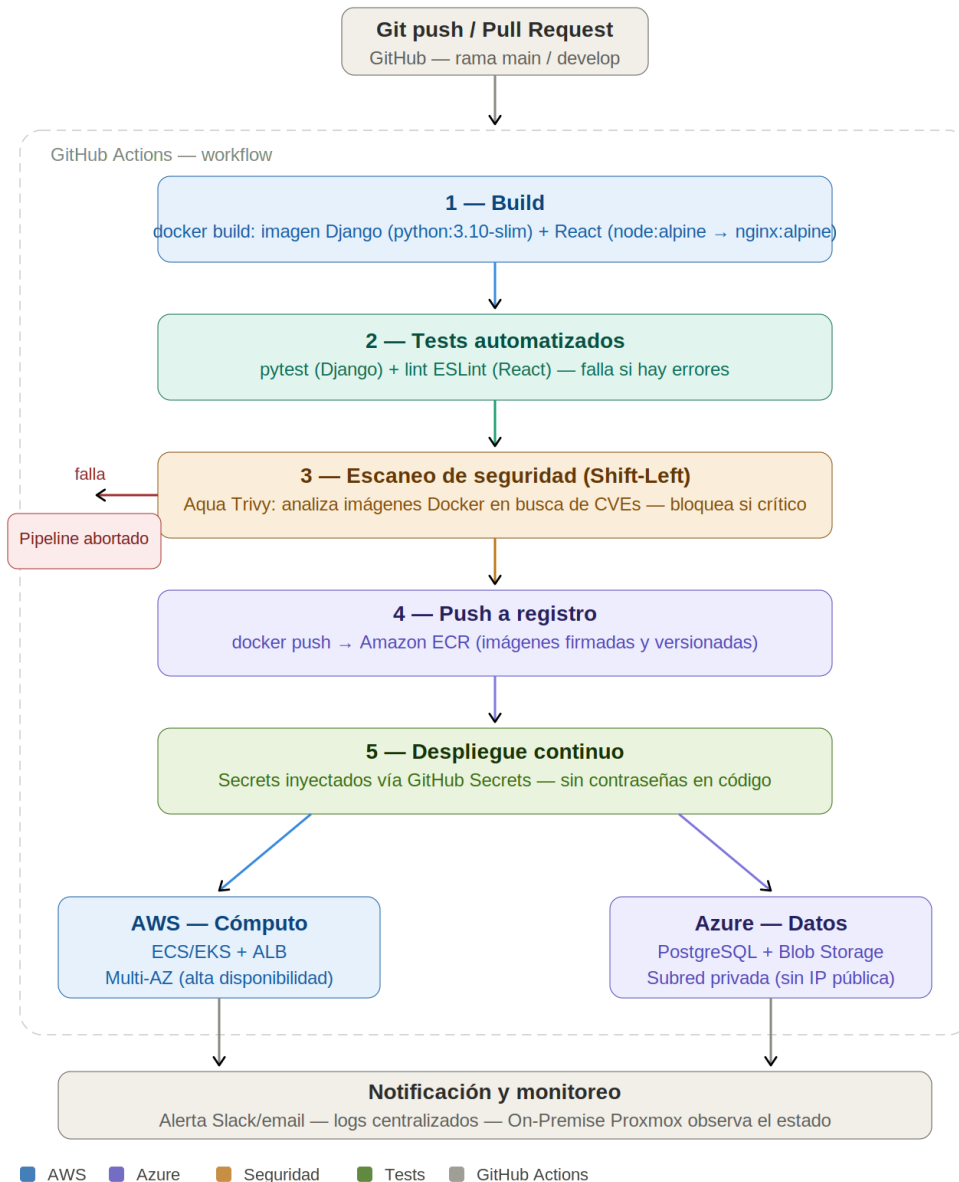
Se propone el uso de Ansible para estandarizar los sistemas operativos de los nodos locales (On-Premise).

Fragmento conceptual — `playbook.yml` para instalar Docker:

```
- hosts: nodos_locales
  become: yes
  tasks:
    - name: Instalar dependencias para Docker
      apt:
        name: ['apt-transport-https', 'ca-certificates', 'curl']
        state: present
    - name: Instalar Docker CE
      apt:
        name: docker-ce
        state: present
```

3.4. Estrategia DevSecOps y Ciberseguridad

A. Flujo de Trabajo CI/CD



Pipeline: GitHub Actions.

El pipeline incluirá las siguientes etapas:

- Construcción: Creación de imágenes Docker para Frontend y Backend.
- Pruebas: Ejecución de pruebas automatizadas del código Django.
- Despliegue: Despliegue continuo hacia los servicios de la nube (AWS y Azure).

B. Seguridad en el Código (Shift-Left)

Se incorporará Aqua Trivy dentro del pipeline de GitHub Actions para escanear el código fuente y las imágenes de Docker en busca de vulnerabilidades antes de la fase de despliegue.

C. Bastionado de Red y Protección de Datos

- Aislamiento: La base de datos Azure PostgreSQL se desplegará en una subred privada, sin asignación de IP pública.
- Reglas NSG/Security Groups: Se configurarán Security Groups en AWS y Network Security Groups en Azure para restringir el tráfico. Solo se aceptará tráfico en el puerto 443 desde internet hacia el balanceador.
- Restricción Backend: El Backend solo aceptará peticiones provenientes del balanceador de carga, nunca directamente desde internet.

D. Gestión de Accesos e Identidad (IAM)

- Menor Privilegio: La comunicación entre los servicios de AWS y Azure se realizará bajo el principio de menor privilegio, otorgando solo permisos de lectura/escritura específicos al bucket y a la base de datos.
- Credenciales Seguras: Se prohíbe el almacenamiento de contraseñas en texto plano en el código; estas serán inyectadas como variables de entorno a través de repositorios seguros de secretos de GitHub Actions.

4. Conclusiones

El diseño arquitectónico propuesto logra un equilibrio óptimo entre modernización y seguridad. La implementación de una estrategia Multicloud (AWS y Azure) combinada con infraestructura On-Premise otorga resiliencia al sistema de la concesionaria.

Los principales desafíos anticipados para la fase de implementación radican en la orquestación correcta de las redes entre ambas nubes públicas y el entorno local. Sin embargo, la integración del modelo DevSecOps desde el primer día, especialmente mediante herramientas como Terraform para IaC y Aqua Trivy para la revisión de imágenes Docker, mitiga los riesgos de configuración y estandariza los despliegues de forma segura y auditable.

5. Bibliografía / Webgrafía

Amazon Web Services (AWS). (2023). AWS Well-Architected Framework. Obtenido de AWS Docs: <https://docs.aws.amazon.com/wellarchitected/>

Microsoft Azure. (2023). Azure Database for PostgreSQL documentation. Obtenido de Microsoft Learn: <https://learn.microsoft.com/en-us/azure/postgresql/>

HashiCorp. (2023). Terraform Registry: AWS and Azure Providers. Obtenido de Terraform Docs: <https://registry.terraform.io/>

Red Hat. (2023). Ansible Documentation: Playbooks and Modules. Obtenido de Ansible Docs: <https://docs.ansible.com/>

Aqua Security. (2023). Trivy: Vulnerability Scanner. Obtenido de Aqua Security Docs: <https://aquasecurity.github.io/trivy/>